

Transaction Tracker Documentation

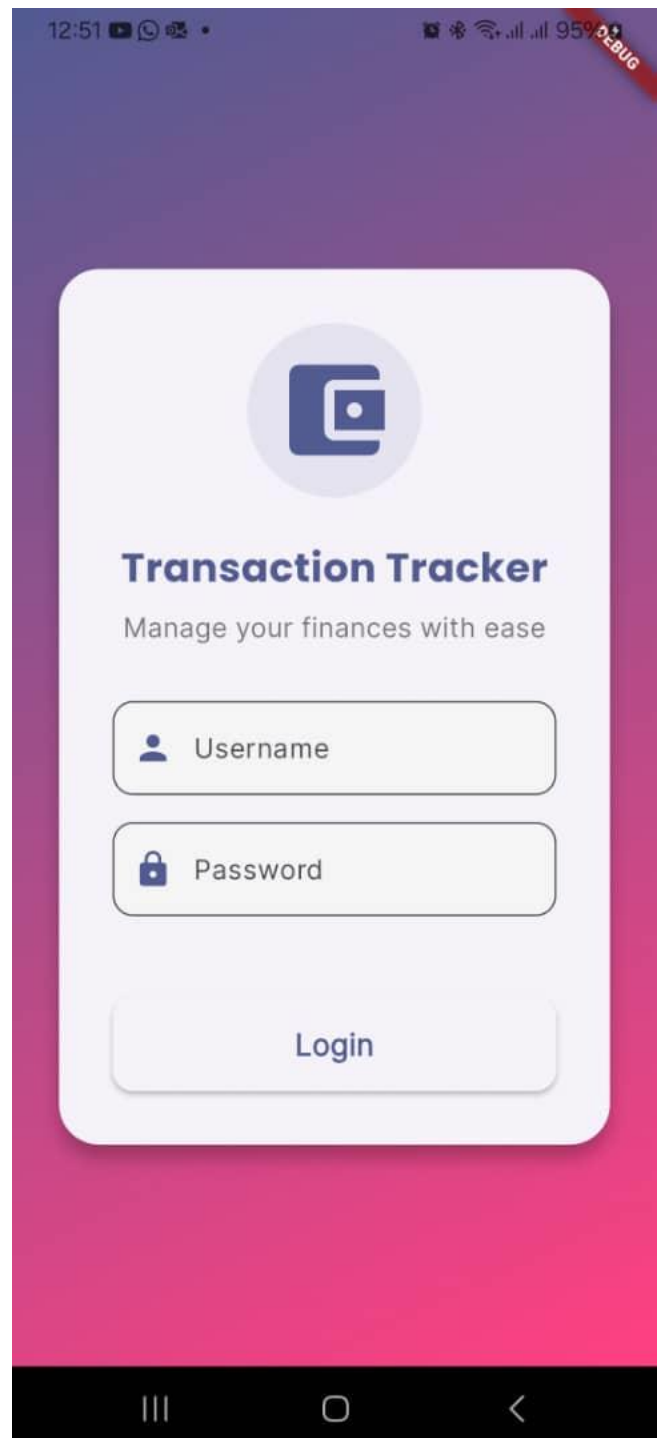
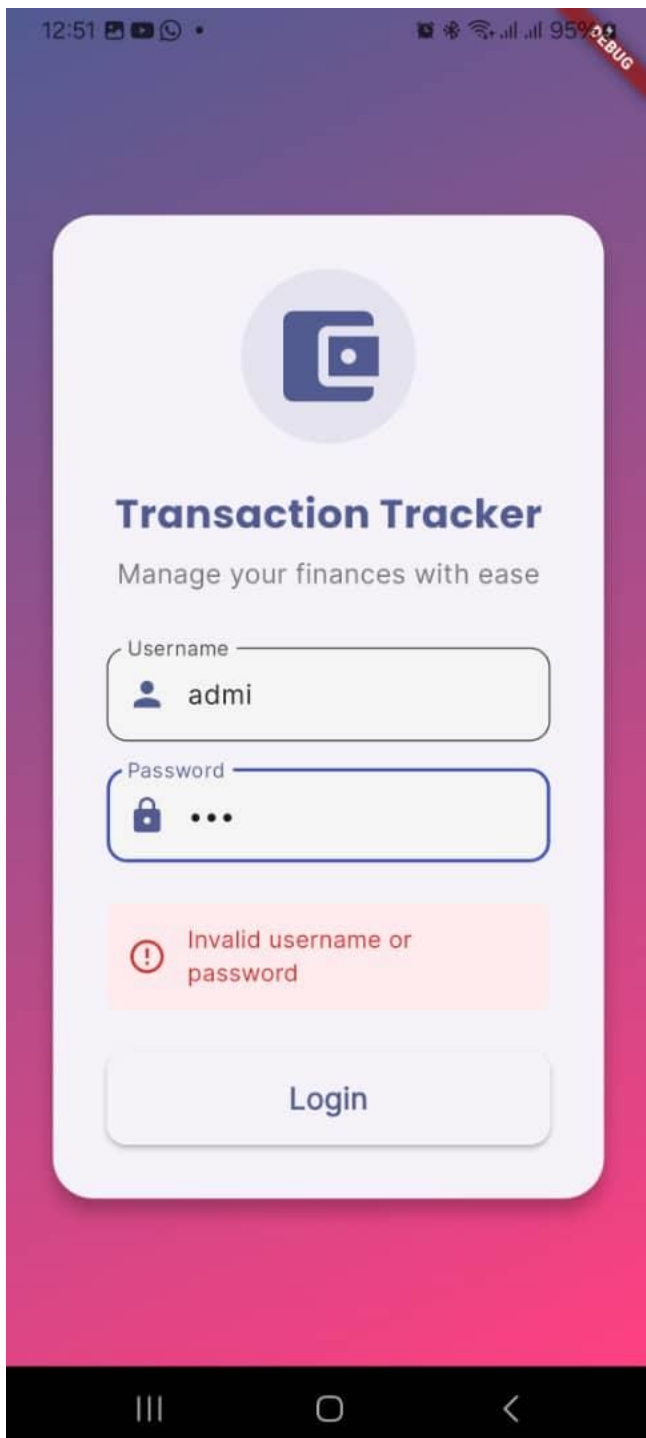
A comprehensive full-stack mobile application for tracking financial transactions, built with Flutter for the frontend and Node.js for the backend. The application provides secure user authentication and real-time transaction management capabilities.

Design Choices and Implementation Challenges

The application was architected with a focus on scalability and maintainability. The Flutter frontend implements a clean architecture pattern with separate layers for UI (screens), business logic (services), and data models.

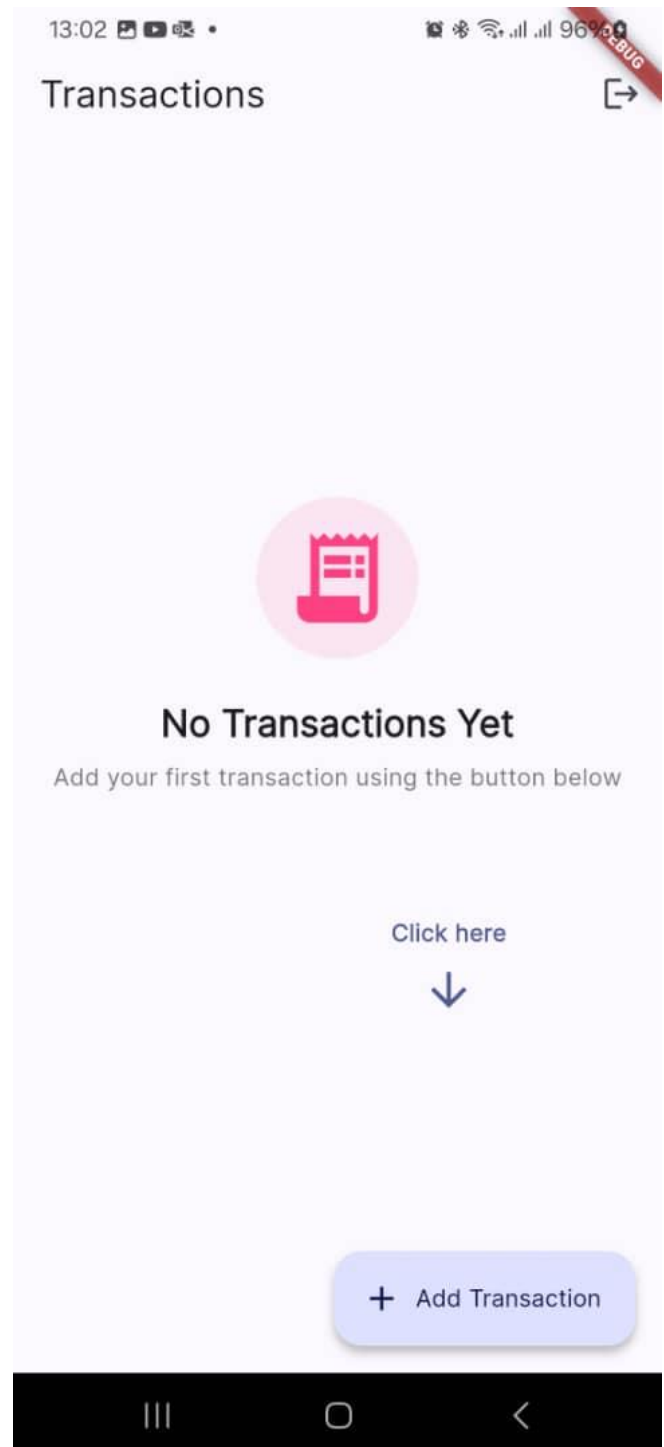
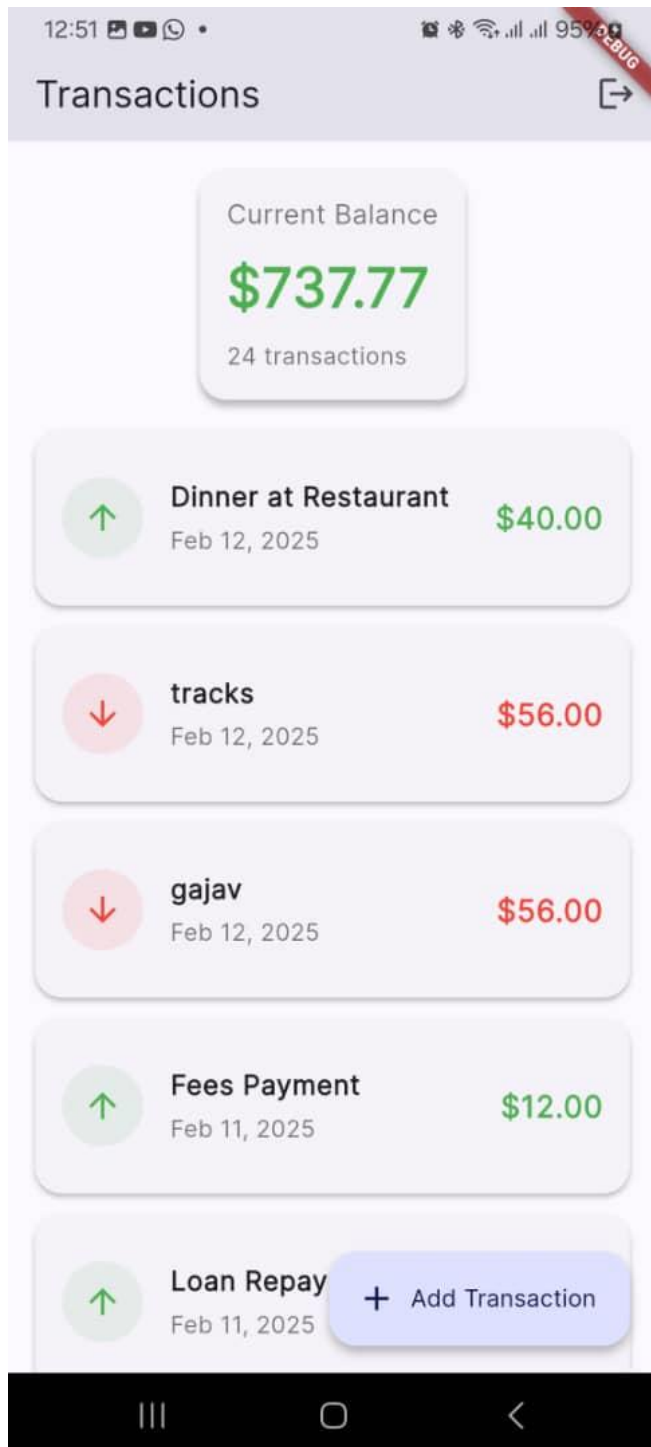
The 3 main screens are as follows:

LOGIN SCREENS



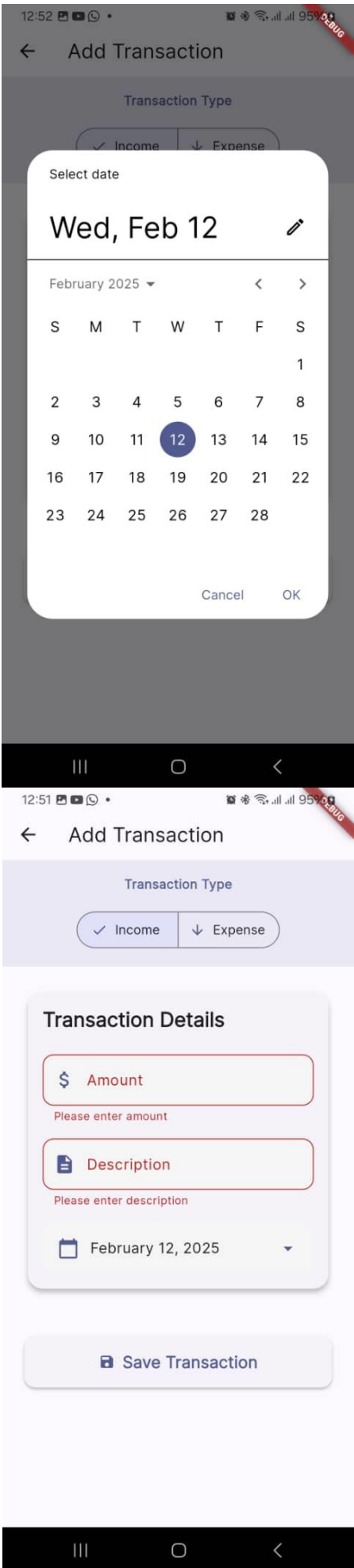
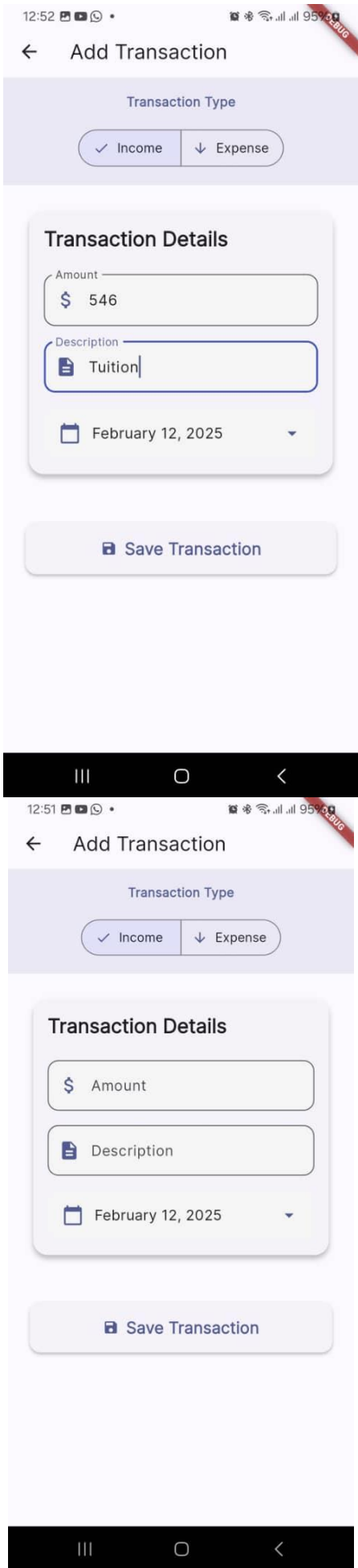
The modern, slick, iconified, colourful and responsive login screen below show what happens when a wrong username and password is entered (**left image above**), and when a user hasn't yet entered anything

TRANSACTIONS SCREEN



The Transaction page shows a list of transactions (**to the left above**), I decided to represent transactions differently i.e. income (**green – up arrow**) and expensive (**red – down arrow**) showing that I have divided transaction. I also added an informative section that shows a message when there are no transactions added yet (**to the right above**). An arrow pointing to the Add transaction button is also added for clarity

ADD NEW TRANSACTION SCREEN



The Add New Transaction Screen allows the user to select transaction type (income and expense) meaning the transaction will be stored as either +/- . The Transaction Details form allow the user to enter an amount, description and date and click Save Transaction button to submit. If the user tries to submit without selecting anything it has validation, meaning it won't submit empty fields. Lastly the default date is set to today's date.

OTHER CONSIDERATIONS

State management is handled efficiently using Provider, while the backend uses Express.js to create a RESTful API with JWT authentication.

Challenges Faced

Key challenges included implementing secure user authentication and maintaining consistent state across the application. These were addressed by creating a robust AuthService that manages JWT tokens and user sessions. Another challenge was ensuring real-time synchronization of transaction data between the frontend and backend, which was solved through careful API design and proper state management.

Prerequisites

- Flutter SDK (latest stable version)
- Java (v17.0.12)

NB. If any other version of Java is used please update the transaction_tracker_app/android/gradle.properties to the path of your java sdk

JVM arguments for Gradle

```
org.gradle.jvmargs=-Xmx4G -XX:MaxMetaspaceSize=2G -
XX:+HeapDumpOnOutOfMemoryError
```

Enable AndroidX and Jetifier

```
android.useAndroidX=true
```

```
android.enableJetifier=true
```

Specify Java compatibility (optional, but recommended)

```
org.gradle.java.home=C:/Program Files/Java/jdk-17
```

Enable parallel project execution (optional, speeds up builds)

```
org.gradle.parallel=true
```

- Node.js (v14 or higher)
- XAMPP (for MySQL database)
- Git
- Code editor (VS Code recommended)